



JavaFX应用的MVC架构

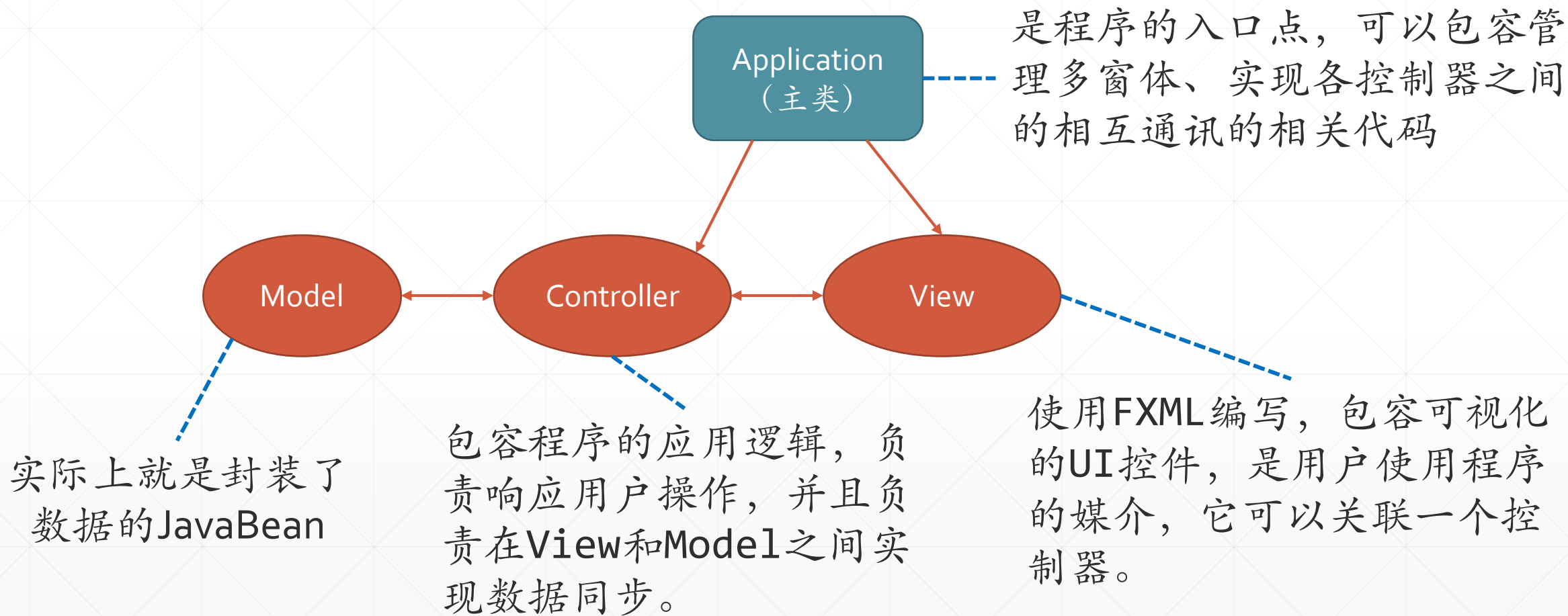
北京理工大学计算机学院
金旭亮

MVC概述



- ✓ MVC是一种设计模式，它的主要目的是将GUI与应用逻辑隔离开来，从而让两者都能独立演化而不会相互影响，避免出现牵一发而动全身的状况。
- ✓ MVC设计模式在许多应用领域得到了广泛应用，有许多软件开发框架是基于MVC设计模式而构建的。
- ✓ JavaFX在设计之初就考虑了应用MVC设计模式，整个JavaFX应用框架可以围绕着MVC设计模式而展开。

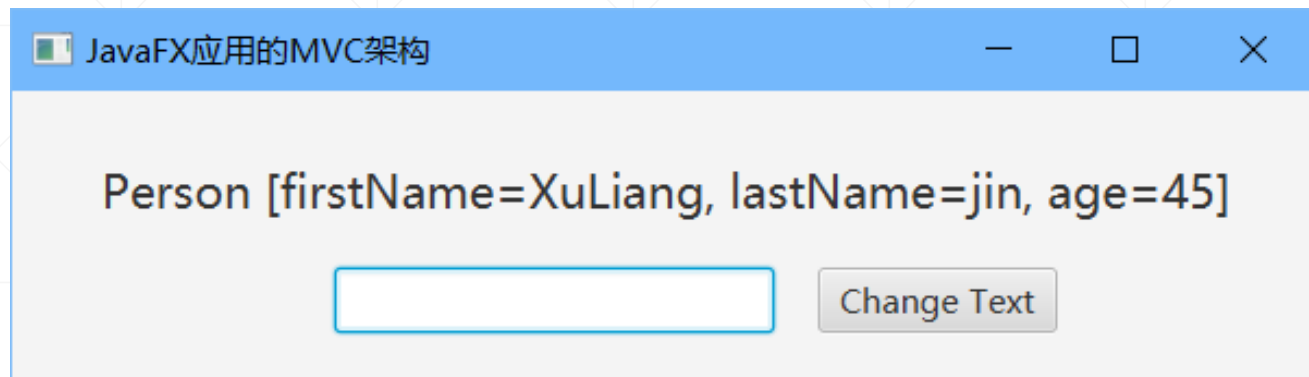
构成JavaFX应用的主要角色





通过实例了解MVC架构

本讲示例：JavaFXMVCDemo



此示例展示了最简单的MVC模式，示例中编写了一个Person类代表Model，PersonController类中包容代码，将Person对象包容的数据显示在View（即窗体）上，同时，PersonController类中也包容按钮控件的事件响应代码，可以直接访问并更新UI控件的值。

示例中的视图

视图文件中，使用fx:controller关联一个控制器

```
<AnchorPane prefHeight="133.0" prefWidth="600.0"
  xmlns:fx="http://javafx.com/fxml"
  fx:controller="com.jinxuliang.javafxmvdemo.PersonController">
  <children...>
</AnchorPane>
```

需要被控制器访问的控件，
需要指定id值

```
<Label fx:id="label"
  alignment="CENTER" layoutY="32.0" text="Hello,JavaFX!"
  AnchorPane.leftAnchor="0.0" AnchorPane.rightAnchor="0.0">
  <font...>
</Label>
```

视图中的元素，通常对应着一个JavaFX对象，在fxml文档装载时实例化。

```
<Button layoutX="300.0" layoutY="210.0"  
    mnemonicParsing="false"  
    onAction="#handleButtonClick"  
    text="Change Text"/>
```

可以指定控件的事件响应方法（放在控制器中）

```
public class PersonController {  
    //获取关联FXML文档中指定Id的控件引用  
    //变量名就是FXML文档中的控件id值  
    3 usages  
    @FXML private Label label;  
    3 usages  
    @FXML private TextField field;  
  
    //此处定义的方法，可以在FXML文档中引用  
    1 usage  
    @FXML  
    public void handleButtonClick() {  
        String text = field.getText();  
        label.setText(text);  
        field.clear();  
    }  
}
```

控制器中，可以注入在FXML中所定义的控件对象，从而在内部访问这些对象。这些对象，由标注有@FXML注解的字段引用。

以@FXML标注的控制器方法，可作为FXML文档中控件对象的事件响应方法。

@FXML注解的作用



@FXML注解会通知JavaFX运行时引擎，它所标记的字段或方法，将会与fxml文档中的元素相关联。



当JavaFX应用启动时，FXMLLoader会装载fxml文档，实例化里面的所有控件，将这些控件装配起来，构成一棵控件树，其根节点被传给Scene，成为Scene Graph。同时，将控制器实例中使用@FXML标识的字段与对应的控件对象关联起来，使用@FXML标记的方法与特定控件对象的特定方法关联起来，从而完成整个JavaFX应用的初始化过程。

//控制器中，可以封装Model作为视图的数据源

4 usages

```
private Person person;
```

no usages

```
public Person getPerson() { return person; }
```

no usages

```
public void setPerson(Person person) { this.person = person; }
```

Person对象的属性为视图提供要显示的数据

控制器中包容视图数据源，并且可以访问Application对象

//可以将Application对象引入注入到控制器中，以便控制器

//能访问其中定义的方法，完成特定的功能（比如切换窗体内容）

1 usage

```
private PersonApplication application;
```

1 usage

```
public void setApplication(PersonApplication app) {
```

```
    this.application = app;
```

```
    person = new Person( firstName: "XuLiang", lastName: "jin", age: "45");
```

```
    label.setText(person.toString());
```

```
}
```

需要时，各控制器之间可以通过Application对象互通信息

Application类

```
public class PersonApplication extends Application {  
    @Override  
    public void start(Stage stage) {  
        this.primaryStage = stage;  
        mainWindow();  
    }  
  
    4 usages  
    private Stage primaryStage;  
  
    //显示主窗体  
    1 usage  
    public void mainWindow() {...}  
  
    public static void main(String[] args) {  
        launch();  
    }  
}
```

程序入口点

Application类中通常是程序入口点，也可以包容一些整个应用程序都可用的功能代码（比如切换窗体显示，或者是完成某些初始化工作）。

在需要的时刻，Application类可以成为各个控制器之间相互通讯的媒介。主要原因在此通常是由它完成创建整个程序的初始化任务，因此，可以方便地访问到各种对象。

Application类中沟通控制器与视图的典型代码

```
//显示主窗体
1 usage
public void mainWindow() {

    try {
        //装载视图
        FXMLLoader loader = new FXMLLoader(
            PersonApplication.class.getResource( name: "person-view.fxml"));
        AnchorPane root = (AnchorPane) loader.load();

        //获取控制器引用
        PersonController controller = loader.getController();
        //将Application类对象引用传入控制器
        controller.setApplication(this);

        //构建场景和舞台
        Scene scene = new Scene(root);
        primaryStage.setScene(scene);
        primaryStage.setTitle("JavaFX应用的MVC架构");
        primaryStage.show();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

通过FXMLLoader对象的getController()方法获取控制器对象的引用，是一种典型的JavaFX编程套路，注意把握。不过这么做的前提是在fxml文档中已经为root元素使用fx:controller关联了控制器的类名。

多学一点：关于控制器的初始化



JavaFX框架定义了一个`Initializable`接口，如果控制器实现了此接口，那么，当`FXMLLoader`实例化一个控制器时，此接口所定义的`initialize()`方法会被`FXMLLoader`自动调用，可以在此方法中编写初始化UI控件的代码（因为此方法会在所有UI控件都被实例化后调用，所以这个方法里的代码可以安全地访问声明在FXML文档中的UI控件）。

```
public class MainWindowController implements Initializable {  
  
    @Override  
    public void initialize(URL location, ResourceBundle resources) {  
  
        System.out.println("MainWindowController.initialize()方法被调用");  
    }  
  
}
```

小结：控制器相关的编程套路



默认情况下，推荐在fxml文档中以fx:controller的方式声明与此fxml文档相关联的控制器，FXMLLoader在装载fxml文档时，会一并实例化控制器并且建立好@FXML字段和方法引用。



调用FXMLLoader实例的getController()方法获取控制器实例。



在特殊场景下，可能需要动态地装载没有声明控制器的fxml文档，这时，可以调用FXMLLoader实例的setController()方法将一个控制器实例与fxml文档的根节点相关联。但要注意，如果想让控制器中的@FXML字段或方法生效，必须在load()方法装载fxml文档之前调用setController()方法。

本讲小结



本PPT所展示的JavaFX示例虽然简单，但却是开发JavaFX桌面应用程序的最基础最简单的形式，后面很多的技巧，都是在这个基础之上展开的。



学完数据绑定技术之后，可以将MVC架构“升级”为MVVM架构，能够大幅度地简化代码，这部分内容，后面介绍。



课后请仔细地阅读本示例源码，务求弄明白Model、View和Controller是如何相互合作的。